👋 Instruction

Documentation Tabs

Please open the provided documentation file on your laptop/PC. The file contains separate tabs for each data structure:

- Instruction
- Array
- Linked List
    - Singly Link List
    - Doubly Linked List
- Stack
- Queue
- Trees
- Sorting Algorithms

Each tab contains specific requirements and examples for the corresponding data structure implementation

General Requirements

Students are required to implement comprehensive programs covering the following core Data Structure topics:

- Arrays
- Linked Lists
- Stacks
- Queues
- Trees
- Sorting Algorithms

Each implementation must be menu-driven, allowing users to perform all standard operations relevant to each data structure.

Implementation Requirements

1. Full Implementation Required

Each program must include all standard operations pertinent to the respective data structure:

- Arrays: Insertion, deletion, display, etc.
- Stacks: Push, pop, peek operations
- Queues: Enqueue, dequeue, display functions
- Linked Lists: Insertion (beginning, middle, end), deletion, traversal
- Sorting: Multiple algorithms (Bubble, Insertion, Selection)

2. Menu-Driven Format

All programs must provide a clear, user-friendly menu interface. Example for Array operations:

===== ARRAY OPERATIONS MENU =====
1. Push to Stack
2. Pop from Stack
3. Peek Stack
4. Enqueue to Queue
5. Dequeue from Queue
6. Display Array
7. Delete Element (Pointer-Based)
8. Find Third Minimum Value
9. Eliminate Duplicates
10. Exit

3. File Structure

Submit one code file per topic, containing all related operations for that data structure. Use the following naming convention:

- <rollno>_Array.cpp
- <rollno>_Stack.cpp
- <rollno>_LinkedList.cpp
- <rollno>_Queue.cpp

Important Note: For each data structure, implement a single comprehensive program with different functions for each operation, integrated through the menu system.

4. Code Standards

- Include explanatory comments throughout your code
- Follow proper indentation and formatting practices
- Use meaningful variable and function names
- Implement appropriate error handling

Documentation Requirements

Dry Run Documentation

A manual dry run of each implementation is mandatory:

- Document step-by-step execution processes
- Record memory/stack/queue state changes
- Track variable values and transitions
- Include your full name and roll number on each page
- Submit clear images or scans compiled into a single PDF named <rollno>_DryRun.pdf or you can upload a dry run PDF for each data-structure.

Important: A sample dry run document (sampleDryRun.pdf) has been provided as reference. You must follow the same step-by-step

Evaluation Process

Viva Examination:

Viva will be conducted to evaluate your understanding.

Be prepared to:

- Explain your implementation logic in detail
- Justify your dry run steps
- Demonstrate understanding of pointer usage and operations
- Address questions regarding performance considerations

Note: The assignment grade will be determined solely through the viva performance.

😆 Array

1. Remove Duplicates from Sorted Array

You are given a sorted array nums. Remove all duplicates **in-place** (i.e., without using another array) so that each element appears only once.
Return the new length of the array after removing duplicates.

Example:

Input: nums = [1, 1, 2, 2, 3, 3, 3, 4]
Output: [1, 2, 3, 4] (new length = 4)

2. Longest Consecutive Sequence:

Given an unsorted array of integers, find the **length of the longest sequence of consecutive numbers** (numbers that appear one after another in order, not necessarily adjacent in the array).

Example:

Input: nums = [100, 4, 200, 1, 3, 2]
Output: 4
Explanation: The consecutive sequence is [1, 2, 3, 4].

3. 3Sum (Find Triplets that Sum to Zero)

Find all unique triplets (a, b, c) in the array such that a + b + c = 0.

Example:

Input: nums = [-1, 0, 1, 2, -1, -4]
Output: [[-1, 0, 1], [-1, -1, 2]]

4. Move Zeroes

Move all zeros to the end of the array while keeping the order of non-zero elements **the same**.

Example:

Input: nums = [0, 1, 0, 3, 12]
Output: [1, 3, 12, 0, 0]

5. Majority Element

Find the element that appears **more than n/2 times** in the array (it always exists).

Example:

Input: `nums = [2, 2, 1, 1, 1, 2, 2]`
Output: 2

Hint: You can use the **Boyer–Moore Voting Algorithm**. Keep a candidate and a counter. Increase counter when the same element appears, decrease otherwise.

6. Subarray Sum Equals K

Given an array of integers `nums` and an integer `k`, count the total number of **continuous subarrays** whose sum equals `k`.

Example:

Input: `nums = [1, 1, 1], k = 2`
Output: 2
Explanation: Subarrays `[1,1]` (indices 0-1) and `[1,1]` (indices 1-2) both sum to 2.

16. Longest Subarray with Sum ≤ K

Find the **length** of the longest subarray whose sum is **less than or equal to K**.

Example:

Input: `nums = [2, -1, 2, 3, -4, 2], k = 3`
Output: `5`
E**xplanation:** The longest subarray `[2, -1, 2, 3, -4]` has sum `2`.

🙂 LinkList

Implement the basic operations of the linklist.

1. Insertion
2. Deletion
3. Updation
4. Searching
5. Etc etc.

After implementing the basic operations of linklist (both singly and doubly), you can move on to the questions part (sub-tabs of the LinkList tab)

Each type of LinkList has different questions.

Singly LinkList

1. Reverse Linked List

Reverse a singly linked list and return the new head. You must reverse it **in-place** (no arrays or extra list).

Example:
Input: 1 → 2 → 3 → 4 → 5
Output: 5 → 4 → 3 → 2 → 1

2. Merge Two Sorted Lists

Given two sorted linked lists, merge them into one sorted list.

Example:

Input:

List1 = 1 → 3 → 5
List2 = 2 → 4 → 6
Output: 1 → 2 → 3 → 4 → 5 → 6

3. Middle of the Linked List

Find the middle node of a singly linked list. If there are two middle nodes, return the second one.

In even nodes, you can consider any node from two as middle.

Example:

Input: 1 → 2 → 3 → 4 → 5
Output: 3

Input: 1 → 2 → 3 → 4 → 5→6
Output: 3 or 4

4. Remove Nth Node from End

Remove the **nth node from the end** of a linked list.

Example:
Input: `head = [1,2,3,4,5], n = 2`
Output: `[1,2,3,5]`

5. Linked List Cycle Detection

Detect if a linked list contains a cycle (loop).

Example:
Input: `head = [3, 2, 0, -4]`, tail connects to node index 2
Output: `true`

Input: `head = [3, 2, 0, -4]`, (3 → 2 → 0 → -4 → NULL)
Output: `false`

7. Add Two Numbers (Linked Lists)

Each linked list represents a number in reverse order. Add the two numbers and return the sum as a linked list.

Example:
Input:

`L1 = 2 → 4 → 3  (represents 342)`

`L2 = 5 → 6 → 4  (represents 465)`

Output: `7 → 0 → 8  (represents 807)`

8. Reorder List

1. Find the middle node.
2. Reverse the second half.
3. Merge both halves alternately.

Example:
Input: 1 → 2 → 3 → 4 → 5
Output: 1 → 5 → 2 → 4 → 3

9. Remove Duplicates from Sorted Linked List

Given a **sorted** linked list, delete all duplicate elements so that each element appears only once.

Example:
Input: 1 → 1 → 2 → 3 → 3
Output: 1 → 2 → 3

10. Reverse Nodes in k-Group:

Given the head of a singly linked list, reverse the nodes of the list **k at a time** and return the modified list. Nodes that remain fewer than k at the end should stay as they are (not reversed). Example:

Input: head = [1, 2, 3, 4, 5], k = 2

Output: [2, 1, 4, 3, 5]

NOTE: Your code should work for any value of K.

Doubly LinkList

1. Insert a Node in a Sorted Doubly Linked List

Insert a new node in a sorted doubly linked list while keeping it sorted.

Example:
Input: 2 ⇄ 4 ⇄ 8 ⇄ 10, value = 6
Output: 2 ⇄ 4 ⇄ 6 ⇄ 8 ⇄ 10

2. Delete All Occurrences of a Given Key

Delete every node that contains the given key.

Example:
Input: 1 ⇄ 2 ⇄ 2 ⇄ 3 ⇄ 4, key = 2
Output: 1 ⇄ 3 ⇄ 4

Hint: Carefully handle deletion at the head, middle, and tail nodes.

3. Find Pairs with a Given Sum

Find all pairs in a sorted doubly linked list whose sum equals a given number.

Example:
Input: 1 ⇄ 2 ⇄ 4 ⇄ 5 ⇄ 6 ⇄ 8 ⇄ 9, sum = 7
Output: (1,6), (2,5)

Hint: Use two pointers: one from the start and one from the end. Move them based on the sum comparison.

4. Rotate Doubly Linked List by N Nodes

Rotate the list by N nodes from the beginning.

Example:
Input: 10 ⇄ 20 ⇄ 30 ⇄ 40 ⇄ 50, N = 2
Output: 30 ⇄ 40 ⇄ 50 ⇄ 10 ⇄ 20

Hint: Find the Nth node, adjust head and tail pointers accordingly.

5. Merge Two Sorted Doubly Linked Lists

Merge two sorted DLLs into one sorted list (without creating new nodes).

Example:
Input: List1 = 2 ⇄ 4 ⇄ 8, List2 = 1 ⇄ 3 ⇄ 5
Output: 1 ⇄ 2 ⇄ 3 ⇄ 4 ⇄ 5 ⇄ 8

Hint: Compare nodes from both lists one by one, adjusting prev and next pointers.

6. Convert Binary Tree to Doubly Linked List

Convert a binary tree into a doubly linked list using inorder traversal.

Example:

Binary Tree:

```
    10
    /  \
   5   20
```

Doubly Linked List: 5 ⇄ 10 ⇄ 20

7. Find Triplets with Given Sum in a Sorted Doubly Linked List

Find all triplets in a sorted doubly linked list that sum to a given value X.

Example:

Input: 1 ⇄ 2 ⇄ 4 ⇄ 5 ⇄ 6 ⇄ 8 ⇄ 9, sum = 17
Output: (2, 6, 9) and (4, 5, 8)

8. Merge K Sorted Doubly Linked Lists

You are given K sorted doubly linked lists.
Merge them all into one sorted list in O(N log K) time.

Example: List1 = 1 ⇄ 4 ⇄ 5, List2 = 1 ⇄ 3 ⇄ 4, List3 = 2 ⇄ 6

Output: 1 ⇄ 1 ⇄ 2 ⇄ 3 ⇄ 4 ⇄ 4 ⇄ 5 ⇄ 6

9. Reverse Doubly Linked List in Groups of K

Reverse nodes of a doubly linked list in groups of size K.

Example:
Input: 1 ⇄ 2 ⇄ 3 ⇄ 4 ⇄ 5 ⇄ 6 ⇄ 7 ⇄ 8, K = 3
Output: 3 ⇄ 2 ⇄ 1 ⇄ 6 ⇄ 5 ⇄ 4 ⇄ 7 ⇄ 8

Hint: Reverse the first K nodes, then recursively call for the rest of the list. Reattach using the new head and tail pointers.

10. Split a Circular Doubly Linked List into Two Halves

Given a **circular DLL**, split it into two equal halves.

Example:
Input: 10 ⇄ 20 ⇄ 30 ⇄ 40 ⇄ 50 ⇄ (back to 10)
Output: List1 = 10 ⇄ 20 ⇄ 30, List2 = 40 ⇄ 50

🙃 Stack

1. Implement a Stack using Array

Implement a stack that supports:

- `push(x)` → add element
- `pop()` → remove top element
- `peek()` → view top element
- `isEmpty()` → check if empty

Handle **stack overflow** and **underflow** conditions.

2. Implement a Stack using Linked List

Build a stack using linked nodes instead of arrays.
Each node should contain `data` and `next`.
Perform push, pop, and display operations.

3. Reverse a Stack using Recursion (No Loops)

Reverse a given stack using recursion only (no arrays or loops).

Example:
Input: `[1, 2, 3, 4]`
Output: `[4, 3, 2, 1]`

4. Sort a Stack using Recursion

Sort a stack in ascending order using only **stack operations** and **recursion**. No loops or extra stack allowed.

Example:
Input: `[3, 5, 1, 4, 2]`
Output: `[1, 2, 3, 4, 5]`

5. Delete Middle Element of a Stack

Remove the middle element of a stack without using another data structure.

Example:
Input: [1, 2, 3, 4, 5]
Output: [1, 2, 4, 5]

Hint: Use recursion to pop until the middle is reached, then rebuild.

6. Check for Balanced Parentheses

Check if every opening bracket has a matching closing bracket.

Example:
Input: {[()()]} → Balanced
Input: {[(])} → Not Balanced

7. Next Greater Element

Find the next greater element for each element in an array. If no such number exists, use -1.

Example:
Input: [4, 5, 2, 10]
Output: [5, 10, 10, -1]

8. Check if Stack Elements are Palindrome

Check if the elements in a stack form a palindrome when read from top to bottom.

Example:

Input: [1, 2, 3, 2, 1]
Output: Palindrome

😁 Queue

1. Implement a Queue using Array

Implement a queue using arrays supporting:

- `enqueue(x)`
- `dequeue()`
- `front()`
- `isEmpty()` and `isFull()`

Handle **overflow** and **underflow** cases.

2. Implement a Queue using Linked List

Implement a queue using a linked list.

Each node contains `data` and `next`.

Maintain **front** and **rear** pointers for efficient enqueue/dequeue.

3. Implement a Priority Queue using Array

Implement a queue where each element has a priority.
The highest-priority element is dequeued first.

Hint:

Store elements with `(data, priority)` pairs.
On dequeue, find and remove the element with the highest priority.

4. Reverse First K Elements of a Queue

Reverse the first `k` elements of a queue while leaving the rest in the same order.

Example:
Input: `Queue = [1, 2, 3, 4, 5]`, `k = 3`
Output: `[3, 2, 1, 4, 5]`

5. Check if a Queue is a Palindrome

Given a queue of integers, check if it reads the same forward and backward. You can use a stack or a temporary queue.

Example:
Input: `[1, 2, 3, 2, 1]`
Output: `Palindrome`

😝 Trees

Implement all the basic operations:

1. Insertion
2. Deletion
3. Updation
4. Searching
5. Traversal (Preorder, Inorder, Postorder
6. Etc etc

BST

1. Find Height of a Binary Tree

Find the maximum depth (height) of a tree using recursion.

Example:

Input:



Output: 3

2. Count Leaf and Non-Leaf Nodes

Count how many nodes are **leaves** (no children) and how many are **non-leaves**.

4. Level Order Traversal

Traverse the tree level by level using a queue.

Example:

```
 Input:

   1

   / \

 2    3


Output: 1 2 3
```

5. Diameter of a Binary Tree (Leetcode #543)

Find the **longest path** between any two nodes in a tree.

6. Check if a Binary Tree is a BST

Verify if all nodes satisfy BST properties.

7. Find Path from Root to a Given Node

Given a node value, print the path from root to that node.

8. Sum of All Left Leaves

Find the sum of all **left leaf nodes** in a binary tree.

9. Print Ancestors of a Given Node

Given a node x, print all its **ancestors** from root to parent.

Example:

Input:

```
      1
     / \
    2   3
   / \
  4   5
```

Ancestors of 5: 2, 1

10. Find Kth Smallest Element in a BST

Return the **Kth smallest** element using inorder traversal.

Example:

Input: `[3, 1, 4, null, 2]`, `k = 1`

Output: `1`

AVL

1. Implement Insertion in an AVL Tree

Write a program to insert nodes into an AVL tree while maintaining its **balance factor**.
Perform necessary **rotations** (LL, RR, LR, RL) to keep it balanced.

Example:
Insert sequence: `10, 20, 30`
After inserting 30 → perform **Left Rotation** → Balanced Tree:

2. Implement Deletion in an AVL Tree

Delete a node from an AVL tree and rebalance the tree if required.
Test all four imbalance cases after deletion.

Example:
Delete `10` from:

```
   20
   /  \
  10  30
      \
       40
```

Perform **Left Rotation** after deletion.

3. Find Balance Factor of Every Node

For each node in the AVL tree, calculate and print the **balance factor** (Left Height − Right Height).

Example:
If for node 20, `Left = 2, Right = 1`
Balance Factor = `+1`

4. Perform All Four Rotations (LL, RR, LR, RL)

Demonstrate each type of rotation with examples.

Example:

- LL Rotation → Right Rotation
- RR Rotation → Left Rotation
- LR Rotation → Left-Right Rotation
- RL Rotation → Right-Left Rotation

Note: You should manually **dry run** each case.

5. Check if a Given Tree is AVL Balanced

Write a function to check if a binary tree satisfies AVL properties.
(Height difference ≤ 1 for every node.)

6. Construct an AVL Tree from a Given Sorted Array

Given a **sorted array**, build a **balanced AVL tree** from it.

Example:
Input: [1, 2, 3, 4, 5, 6, 7]
Output:

```
     4
    / \
   2   6
  / \ / \
 1  3 5  7
```

This is a perfectly balanced AVL tree.

Note: When a user selects this option, you've to ask about the size of the array and then create an array of that size. Ask for values from the user. And then pass that array to this function and return/construct an AVL tree.

7. Print the AVL Tree in Level Order

Perform **BFS traversal** using a queue to print the AVL tree level-by-level. It visually verifies if the tree is balanced.

Example:
For tree:

```
   30
   /  \
  20  40
 /      \
10       50
```

Output: 30 20 40 10 50

😏 Algorithms

Searching

1. Linear Search

Write a program to search for an element in an array using linear search.
Display the index if found, otherwise print “Not Found”.

2. Binary Search (Iterative)

Implement binary search in a sorted array using an iterative approach.

Example:
Input: `[1, 2, 3, 4, 5, 6, 7]`, target = 4 → Output: `Found at index 3`

3. Binary Search (Recursive)

Perform binary search using recursion. Explain base and recursive cases clearly.

Insertion

1. Implement Bubble Sort

Sort an array using bubble sort and print the array after each pass.

Hint:
Compare adjacent elements and swap if needed.

2. Implement Selection Sort

Find the smallest element and place it at the beginning, repeat for all positions.

3. Implement Insertion Sort

Sort an array by inserting elements into the correct position of the already-sorted part.

4. Implement Merge Sort

Divide array into halves, sort recursively, and merge sorted halves.

Hint:
This uses **Divide and Conquer**.

5. Implement Quick Sort

Choose a pivot, partition the array, and recursively sort left and right partitions.

Hint:
Best case: O(n log n), Worst case: $O(n^2)$

6. Implement Heap Sort

Use a **Min Heap or Max Heap** to sort elements.

Steps:

- Build heap
- Extract root repeatedly
- Re-heapify

7. Implement Counting Sort

Use frequency counting to sort non-negative integers.

Hint:
Efficient for small integer ranges.

😑 Recursion

1. Print All Subsets of a Given Set (Power Set Problem)

Example:
Input: `{1, 2, 3}`
Output: `{}`, `{1}`, `{2}`, `{3}`, `{1,2}`, `{1,3}`, `{2,3}`, `{1,2,3}`

2. Generate All Permutations of a String

Example:
Input: `"ABC"`
Output: `"ABC", "ACB", "BAC", "BCA", "CAB", "CBA"`

Hint: Swap characters and recursively fix the rest of the string.

3. Subset Sum Problem

Determine if there exists a subset of numbers that sums to a given target.

Example:
Input: `{3, 34, 4, 12, 5, 2}`, sum = 9
Output: `true` (subset {4, 5})

4. Fibonacci Sequence

Print first `n` Fibonacci numbers.

Example:
Input: `n = 5` → Output: `0 1 1 2 3`

Recurrence: `f(n) = f(n-1) + f(n-2)`

5. Reverse a String

Example:
Input: `"hello"` → Output: `"olleh"`

6. Find the Power (x^n)

Example:
Input: `x = 2, n = 5` → Output: `32`

Logic:
If `n` even → `power(x, n/2) * power(x, n/2)`
If `n` odd → `x * power(x, n-1)`

6. Count Digits of a Number

Example:
Input: `12345` → Output: `5`

7. Binary Search using Recursion

Example:
Input: `[1, 2, 3, 4, 5, 6]`, search = 4 → Output: `Found at index 3`

Logic:
Check mid → if not found, recurse left or right half.

8. Find Maximum Element in Array

Example:
Input: `[3, 9, 1, 7, 5]` → **Output:** `9`

Hint: `max = max(arr[n-1], findMax(arr, n-1))`

9. Count Occurrences of a Number in Array

Example:
Input: `[1,2,3,2,4,2]`, **target=2** → **Output:** `3`

Hint:
Increment count when match found, then recurse.

10. Reverse an Array using Recursion

Example:
Input: `[1, 2, 3, 4, 5]` → **Output:** `[5, 4, 3, 2, 1]`